UNIVERSITY OF PISA

DEPARTMENT OF COMPUTER SCIENCE

Master's Degree in Computer Science

AMM Scheduler:

Design and Implementation of a Runtime Scheduling System for Asymmetric Heterogeneous Computing

Supervisor:

Prof. Marco Danelutto

Candidate:

Simone Stanganini

ACADEMIC YEAR 2024/2025

Contents

Chapter 1

Introduction

1.1 Contest

Until a few years ago, performance improvements depended mainly on increasing CPU clock speeds, today the approach has changed completely, shifting towards hardware specialization.

Modern computers, from servers to embedded SoCs, don't rely on a single processor anymore, instead they integrate different specific accelerators:

- **CPU**: Optimized for sequential tasks and low latency.
- **GPU**: Available as integrated (iGPU) or discrete (dGPU), in a lot of cases both at the same time, designed for massive parallel computing.
- **NPU** (Neural Processing Unit): A recently introduced unit, specialized in speeding up matrix operations for AI workloads.

This variety of hardware has created a gap with the software, while hardware offers different resources for different workloads, traditional software tends to be static. It usually assigns tasks to a single device ,often just the CPU or GPU, and ignores the other available resources.

The result is an inefficient use of the system. We could get more throughput with the same efficiency, or be more efficient at the same speed, so the current challenge is *orchestration*: we need a system that can dynamically map each task to the right accelerator, using the entire heterogeneous topology of the system.

1.2 Objectives

The main goal of this thesis was to design and implement a scheduler capable of dynamically assigning computational tasks, specifically streams of matrix multiplications, to the different accelerators found in modern architectures.

The work started with an essential preliminary phase, I focused on integrating and studying different compute backends, such as CUDA, SYCL, OpenVINO. This step was useful to create a uniform abstraction layer over the hardware and to verify the technical feasibility of the system.

After this initial phase, the work moved on to the actual development of the scheduler, the core of this project focused on the asymmetric orchestration logic, designed to decide in real-time which resource to use for each specific workload for maximize utilization and efficiency.

Finally, the system was validated with experimental results on a complex hybrid architecture, this setup included an Intel multicore CPU (with integrated GPU and NPU) and a discrete NVIDIA GPU. The tests used to verify the operation were streams of identical matrices, streams of heterogeneous matrices, and splitting larger tasks across multiple accelerators.

1.3 Thesis Structure

The work is organized into the following chapters:

- **Chapter 2**: Introduces the hardware technologies (CPU, GPU, NPU), the software backends used (CUDA, SYCL, OpenVINO, OpenBLAS) and the software stack used for this project (C, C++, Cmake).
- **Chapter 3**: Describes the logical design, divided into two phases: the creation of the benchmarking environment and the subsequent design of the scheduler.
- **Chapter 4**: Details the code implementation, it covers the wrappers for hardware abstraction, the essential data structures for the scheduler (ringbuffer and performance map), and the techniques used for task splitting.
- **Chapter 5**: Analyzes the experimental results, validating the effectiveness of the scheduler on different workloads.
- **Chapter 6**: Draws conclusions on the work done and suggests possible future developments.

Chapter 2

Tools and Technologies

This chapter presents the hardware and software resources used to develop and validate the system. The choice to work on a hybrid and highly heterogeneous architecture was made to test the scheduler in a real-world scenario, and in this scenario, devices with different performance characteristics and programming models have to work together.

2.1 Experimental Platform

All development and benchmarking activities were done on a mobile workstation Lenovo Yoga Pro 9i gen 9. This platform represents a modern high-performance "Edge" computing node well, it integrates all the types of accelerators studied into a single physical system: multicore CPU, NPU, integrated GPU, and discrete GPU, and having these components available at the same time allowed to emulate, within a single node, the orchestration issues of heterogeneous distributed systems.

2.1.1 The SoC: Intel Core Ultra 9 185H

The core of the system is the Intel Core Ultra 9 185H processor, based on the "Meteor Lake" architecture [2]. This component marks a significant change from traditional monolithic CPUs. It uses a disaggregated design (tiled architecture) that combines different specialized processing units:

Figure 2.1: Intel Ultra Logo

- **Host CPU**: This cpu is made of 16 hybrid cores, 6 Performance-cores, 8 Efficient-cores, and 2 Low-Power-cores for efficiency. In this project this unit acts both as host and accelerator, it runs the scheduler code and does some computation through the OpenBlass library (2.3).
- **NPU**: The Ultra 9 includes a native Neural Processing Unit. This is a low-power accelerator designed specifically for neural network inference. Its compute acceleration is facilitated by Neural Compute Engines, which consist of hardware acceleration blocks for AI operations like Matrix Multiplication and Convolution, alongside Streaming Hybrid Architecture Vector Engines for general computing tasks [3]. In this project the NPU is used via the OpenVINO 2.5 backend to run Matrix Multiplication.
- **iGPU**: Intel Arc Graphics, integrated into the SoC, this graphics unit based on Xe-LPG architecture [1], offers parallel computing capabilities with direct access to system memory, in this project this accellerator has been used with the SYCL library (2.4) as the backend to leverage the full performance of this accellerator.

2.1.2 NVIDIA GeForce RTX 4060

Figure 2.2: Nvidia Rtx Logo

To complement the Intel SoC the system is equipped with a discrete NVIDIA GeForce RTX 4060 Laptop GPU. This device is based on the Ada Lovelace architecture [5] and acts as the high-throughput accelerator of the system.

Unlike the integrated GPU this card features 3072 CUDA Cores for general parallel processing and 96 Fourth-Generation Tensor Cores. The Tensor Cores are specifically designed to accelerate dense matrix multiplications with 16 or even 8 bit operations.

The device utilizes 8 GB of GDDR6 VRAM with a 128-bit memory interface, this dedicated memory provides high bandwidth but requires explicit data transfer over the PCIe bus.

Within the scheduler this device is managed via the CUDA backend 2.2.

2.2 CUDA Toolkit

Figure 2.3: Nvidia Cuda Logo

To exploit the computational capabilities of the discrete NVIDIA GPU the system utilizes CUDA (Compute Unified Device Architecture). This parallel computing platform and programming model developed by NVIDIA that enables dramatic increases in computing performance by harnessing the power of the GPU. It allows developers to accelerate compute- intensive applications using C, C++, and Fortran, and is widely adopted in fields such as deep learning, scientific computing, and high-performance computing (HPC) [6].

In this project CUDA is the tool chosen to extract maximum throughput from the hardware 2.1.2. The implementation relies on specific libraries and mechanisms to optimize the execution of matrix multiplication streams.

- **cuBLAS and Tensor Cores**: to perform linear algebra operations we adopts cuBLAS (CUDA Basic Linear Algebra Subroutines). This library is the GPU-accelerated version of the standard BLAS specification and provides highly optimized implementations of matrix operations, specifically, cuBLAS allows the utilization of Tensor Cores, specialized hardware units designed to accelerate mixed-precision matrix multiplication (Float16 and Int8). This significantly increases the theoretical throughput compared to standard floating-point units.
- **CUDA Streams**: To handle the latency introduced by data transfers between the CPU and the GPU the system utilizes CUDA Streams. A stream is a sequence of operations that execute in issue-order on the GPU, by employing multiple streams it is possible to achieve concurrency: while one stream is transferring data over the PCIe bus another stream can execute a computational kernel [7]. This mechanism, known as "latency hiding", is fundamental for real-time processing and ensures that the GPU compute units remain active as much as possible.
- **NVIDIA Management Library (NVML)**: For the analysis of energy efficiency the project integrates the NVIDIA Management Library (NVML). This serves as a monitoring tool that provides direct access to the GPU hardware sensors, it allows the retrieval of real-time metrics such as power usage (in milliJoule) and temperature without requiring external physical probes.

2.3 OpenBlass

Figure 2.4: OpenBlass Logo

2.4 SYCL

Figure 2.5: SYCL Logo

2.5 Openvino

Figure 2.6: OpenVino Logo

2.6 C/C++

Figure 2.7: C and C++ Logo

The C and C++ programming languages represent two of the most influential and widely used tools in the history of computer science. Known for their performance, versatility, and low-level control, they are employed in a vast range of sectors, from operating systems to high-performance applications.

Below is a brief overview of both languages:

The C Language:

- Developed in the 1970s by Dennis Ritchie, it was one of the first high-level languages in history.
- it is renowned for its simplicity, efficiency, and direct access to memory, making it ideal for writing highly optimized code [4].

The C++ Language:

- C++ is an extension of C developed in the 1980s by Bjarne Stroustrup. Among its key features is the support for Object-Oriented Programming (OOP), which allows for the creation of more structured and modular software compared to procedural C [8].
- It provides a rich Standard Library (STL) and is the industry standard for performance-critical applications.

In the context of this project, C plays a critical architectural role. It was utilized to define the Application Binary Interface (ABI) between the scheduler and the various hardware wrappers. Instead C++ serves as the primary development language. Its adoption was mandatory as it is the native language required by the SDKs of all the utilized frameworks, It is used to implement the complex logic of the scheduler and the data structures necessary for task management. Further details on this will be provided in the Implementation Chapter 4.

2.7 Cmake

Figure 2.8: Cmake Logo

Chapter 3

Logical Design

Chapter 4

Implementation

Chapter 5

Results

Appendix A

Questa è un'appendice

Fusce mauris. Vestibulum luctus nibh at lectus. Sed bibendum, nulla a faucibus semper, leo velit ultricies tellus, ac venenatis arcu wisi vel nisl. Vestibulum diam. Aliquam pellentesque, augue quis sagittis posuere, turpis lacus congue quam, in hendrerit risus eros eget felis. Maecenas eget erat in sapien mattis porttitor. Vestibulum porttitor. Nulla facilisi. Sed a turpis eu lacus commodo facilisis. Morbi fringilla, wisi in dignissim interdum, justo lectus sagittis dui, et vehicula libero dui cursus dui. Mauris tempor ligula sed lacus. Duis cursus enim ut augue. Cras ac magna. Cras nulla. Nulla egestas. Curabitur a leo. Quisque egestas wisi eget nunc. Nam feugiat lacus vel est. Curabitur consectetuer.

Bibliography

[1] Intel Corporation. *Intel Arc Graphics Xe-LPG Architecture*, 2023. https://www.intel.com/content/www/us/en/docs/oneapi/optimization-guide-gpu/2023-2/intel-xe-gpu-architecture.html.

[2] Intel Corporation. *Intel Core Ultra Processors (Series 1)*, 2024. https://www.intel.com/content/www/us/en/products/sku/236849/intel-core-ultra-9-processor-185h-24m-cache-up-to-5-10-ghz/specifications.html.

[3] Intel Corporation. *Intel's Neural Processing Unit*, 2024.

[4] Brian Kernighan and Dennis Ritchie. *The C Programming Language*. Prentice Hall, 1988.

[5] Klaus Hinum. *NVIDIA GeForce RTX 4060 Laptop GPU Spec*, 2022. https://www.notebookcheck.net/NVIDIA-GeForce-RTX-4060-Laptop-GPU-Benchmarks-and-Specs.675692.0.html.

[6] NVIDIA Corporation. *CUDA C++ Programming Guide*, 2024. https://docs.nvidia.com/cuda/cuda-c-programming-guide/.

[7] Steve Rennich. *CUDA C++ Streams and Concurrency*, 2024. https://developer.download.nvidia.com/CUDA/training/StreamsAndConcurrencyWebina

[8] Bjarne Stroustrup. *The C++ Programming Language*. Addison-Wesley, 2013.